

Project 5 — ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker

Title: ContentForge Pro Production Ready Content Pipeline With API Database And Docker

Type: Multi-agent (ContentForge's 5 agents, made production-ready) · **Stack:** Python, LangGraph, FastAPI, SQL, Docker, testing/MLOps · **Level:** Advanced / Capstone **Tagline:** ContentForge, served as a real API — saved, containerized, tested, and defendable under a security/scaling review.

Overview

ContentForge Pro takes ContentForge's finished 5-agent system and turns it into a real, deployable service: served over a REST API, backed by a database, containerized, rate-limited, and gated by an automated test suite before any new version can go live. It solves a real problem: a multi-agent system that only runs when someone opens a terminal cannot be used by a team, called by another service, or trusted with real traffic. No new agents are built here — ContentForge's five agents (Supervisor, Research, Analysis, Writer, Reviewer) are already finished; this project is entirely about running them safely, which is the gap between a tutorial project and a hireable, production-grade one.

Features

- REST API (FastAPI) exposing the ContentForge graph over HTTP with typed request/response models and safe error handling
- Every run persisted to a database and retrievable by ID, with a per-request ID threaded through every log line
- Dockerized deployment that runs identically on any machine, with secrets passed only via environment variables at run time — never baked into the image
- In-memory rate limiting using a rolling request-timestamp window, closing the gap a fixed-reset counter leaves open
- Automated pytest-based test suite (including an LLM-judge check) that gates any new version before it's allowed to go live
- Rollback function that restores the last version that actually passed the test suite, scanning back through failed versions if needed
- Structured, request-tagged logging good enough to debug a bad production answer after the fact

Tech Stack

- Python
- LangGraph
- FastAPI
- SQL (SQLAlchemy or SQLite)
- Docker
- pytest
- OpenAI API

Prerequisites

- Comfortable with FastAPI basics — routes, request/response models
- Know basic database storage (SQL or SQLite)
- Familiar with Docker basics — building and running a container, passing secrets via environment variables
- Comfortable writing automated tests with pytest
- Have a working ContentForge agent system (or an equivalent LangGraph multi-agent graph) ready to wrap in an API — this project doesn't build new agents, it makes an existing one production-ready

Architecture

FastAPI exposes a `POST /run-task` route that calls ContentForge's existing 5-agent LangGraph graph directly — none of the agents themselves change. Every run is written to the database as `"pending"` and then updated with its result, and can be looked up later via `GET /runs/{id}`. The whole app is packaged in a Docker container with secrets injected only at runtime, and protected by rolling-window rate limiting. A pytest-based suite, including an LLM-judge check, must pass before a new prompt or setting version is allowed to go live, and a rollback function restores the last version that actually passed — not just whatever came before it.

Setup (do this once, before Step 1)

```
cd project_5_contentforge_pro
python -m venv .venv
source .venv/bin/activate
pip install openai langchain langchain-openai langgraph fastapi "uvicorn[standard]"
pip freeze > requirements.txt
```

New here: `fastapi` + `uvicorn` (the API layer), `sqlalchemy` (database — or use the built-in `sqlite3` directly, your call, either works fine at this scale), `pytest` (for the automated test suite). You'll also need [Docker installed](#) on your computer for Step 3 — this is a one-time setup on your machine, not a pip package.

Troubleshooting

Problem	Fix
A secret ends up baked into a Docker image layer	Never <code>COPY</code> a <code>.env</code> file into the image; pass secrets in only through <code>docker run -e</code> / environment variables at run time, and check with <code>docker history</code>
An internal error shows a raw error trace to an API client	Catch it broadly at the route level, log the full detail on the server, return a plain, safe message with the right 5xx code
The test suite still passes after a prompt gets worse	The suite has a hole — if a change you made worse on purpose doesn't drop the score, fix the test rules or judge prompt before trusting the suite again
Database connections leak under repeated requests	Use connection pooling / properly closed sessions; recreate the leak under real load before saying it's fixed

How To Build This — Step by Step

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

Unlike a regular build, these steps aren't about building more agents — ContentForge's 5 agents are already finished. Each step here makes the *system around* them safer, one production concern at a time, so a failure at any step can be traced to that one step.

Step 1 — ContentForge, Reachable Over HTTP for the First Time

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 1 of 4: ContentForge, Reachable Over HTTP for the First Time

What this step does: proves ContentForge can be reached over HTTP at all, with none of the other production concerns yet — no database, no Docker, no login. Keeps "does the API layer work" separate from everything else. **Why this step matters:** every later step in this project builds on the assumption that the API layer itself is solid — if a database bug or a Docker bug shows up later, this step is what lets you rule out "is it even the HTTP layer" immediately. **Helpful background:** FastAPI basics.

What to do:

1. Write one FastAPI route, `POST /run-task`, that calls ContentForge's graph directly and returns the result — kept only in memory, nothing saved yet. Wrap the graph call in a try/except that returns a plain, safe error message on failure — a raw Python traceback handed back to an API caller is a real information leak, not just an ugly error page.
2. Add typed request/response Pydantic models, and confirm FastAPI's automatic 422 error on a bad request.
3. Test it by actually running `uvicorn` locally and hitting the route with `curl` or the auto-built `/docs` page — not just reading the code.

What's new: ContentForge is now reachable over HTTP, not just from a terminal script. **What stays the same:** the graph itself — Step 1 doesn't change any agent.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

When you'll hit this for real: the exact moment any project moves from "works on my machine" to "someone else needs to call this" — every real AI engineering job includes this step.

Your files after Step 1:

```
project_5_contentforge_pro_production/
├─ api/
│   ├── main.py          → FastAPI app
│   ├── routes.py        → POST /run-task, GET /health
│   └─ schemas.py        → request/response Pydantic models
└─ (project_4_contentforge's graph, imported, unchanged)
```

Step 2 — Every Run Saved to a Database and Findable by ID

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 2 of 4: Every Run Saved to a Database and Findable by ID

What this step does: makes runs durable and traceable — a run's result now survives a restart and can be looked up later, which is the real requirement for the testing/rollback work in Step 4. **Why this step matters:** Step 4's test suite and rollback both need a real record of what happened on past runs to compare against — without this step's database, there's nothing for either of them to check. **Helpful background:** database storage basics.

What to do:

1. Add a SQLite `Run` table (input, output, log, status, timestamps) and write to it on every request. Write the row as `"pending"` before calling the graph, then update it after — that way a lookup on a run that's still in progress returns an honest "pending" instead of looking like it never happened.
2. Add a `GET /runs/{id}` route to read a run back.

3. Add an ID for each request, threaded through every log line, so one run's logs are easy to search for afterward.
4. Test it: kill the database connection in the middle of a request (a classic "break it on purpose" test) and confirm a clean 500, not a raw error trace.

What's new vs. Step 1: every run is now saved and can be looked up on its own; logs are tagged per request. **What stays the same:** the API from Step 1 — `POST /run-task`'s request/response shape doesn't change, it just now has a side effect (a database write).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

When you'll hit this for real: the first time someone asks "what happened on that run from yesterday" and your honest answer is "I don't know, it's gone" — this step is what prevents that conversation.

Your files after Step 2:

```
project_5_contentforge_pro_production/
├── api/
│   ├── main.py
│   ├── routes.py      → now writes to the database, includes GET /runs/{id}
│   └── schemas.py
├── db/
│   ├── models.py      → the Run table
│   └── database.py
└── (ContentForge's graph, unchanged)
```

Step 3 — Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 3 of 4: Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse

What this step does: makes the app portable and hard to abuse — anyone can run it the exact same way from a container, and one client can't take it down by hammering it with requests. **Why this step matters:** "works on my machine" is not a claim anyone else can verify or rely on — a container plus rate limiting is what turns this from a project only you can run into one anyone can run and trust under real traffic. **What's new vs. Step 2:** a working `Dockerfile` and basic rate limiting get added. **What stays the same:** the API and database schema from Steps 1-2 — this step wraps operational safety around them, it doesn't redesign them. **When you'll hit this for real:** handing your project to anyone else to run — a teammate, a hiring manager, a server that's never seen your code before. "It works on my machine" stops being an acceptable answer here. **Helpful background:** Docker basics.

What to do:

1. Write a `Dockerfile` ; confirm the app runs the same from a container, with only environment variables passed in at `docker run` time — no secret baked into any layer (check with `docker history`).
2. Add basic in-memory rate limiting on `POST /run-task` . Prefer tracking actual request timestamps in a rolling window over a counter that resets on a fixed timer — a fixed-reset counter lets a client send double the intended rate right at the boundary where the counter resets.
3. Test it: the secret-in-image check with `docker history` , a missing-env-var container startup failure, and hitting the rate limit with quick repeated requests.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Your files after Step 3:

```
project_5_contentforge_pro_production/
├── api/
│   ├── main.py
│   ├── routes.py          → now rate-limited
│   └── schemas.py
├── db/
│   ├── models.py
│   └── database.py
├── Dockerfile
└── .dockerignore
```

Step 4 — Test-Gated and Ready to Roll Back: No Regression Ships Quietly (final)

Project: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — Step 4 of 4: Test-Gated and Ready to Roll Back: No Regression Ships Quietly

What this step does: the real "safe to keep changing" layer — a test suite that would catch a bad prompt change, and a release gate that refuses to let it go live. **Why this step matters:** every project up to this point proved things worked *once* — this step is what lets you keep improving ContentForge afterward without each change being a gamble on whether you quietly broke something that used to work. **What's new vs. Step 3:** a test suite and a versioned release gate + rollback get added on top of the containerized app. **What stays the same:** everything from Steps 1-3 — this step doesn't touch the API, database, or Docker setup, it adds a safety net around future changes to them. **When you'll hit this for real:** the day after this project is "done," when you want to change a prompt without risking breaking what's already working — this is the difference between "shipped once" and "safe to keep improving." **Helpful background:** testing, evaluation, and observability basics, plus MLOps/LLMOps concepts (versioning, release gates, rollback).

What to do:

1. Build a test suite against ContentForge's outputs — a fixed set of tasks with clear pass/fail or scored rules.
2. Build a release gate on top of it — a new prompt/setting version can't go "live" without passing the suite.
3. Build a rollback function — goes back to the last version that actually passed, not just "whatever came before." Store every version's suite result, including failed ones, and have rollback scan backward through however many failed versions it takes — the version immediately before the current one isn't necessarily one that ever passed.
4. Test it: a version made worse on purpose gets refused by the release gate, and a rollback correctly restores the last version that passed.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Your files after Step 4 (final):

```
project_5_contentforge_pro_production/
├── api/
│   ├── main.py
│   ├── routes.py
│   └── schemas.py
├── db/
│   ├── models.py
│   └── database.py
├── eval_suite/           → automated test suite
├── versioning.py         → version tracking
├── deploy_gate.py        → release gate
├── rollback.py           → rollback logic
├── Dockerfile
├── .dockerignore
└── test_api.py
```

Final Deliverable: ContentForge-Pro-Production-Ready-Content-Pipeline-With-API-Database-And-Docker — ContentForge's 5 agents, served over FastAPI, saved to a database, containerized with secrets handled correctly, rate-limited, and protected by a test-gated release process with rollback.

Checklist Before You Call This Done (matches the linked docs for full detail)

- ☐ The multi-agent system (ContentForge's 5 agents) served over FastAPI with proper request/response models and error handling
- ☐ A working database for run/conversation history
- ☐ Containerized with a correct Dockerfile — no secrets baked into the image

- ☐ The human-approval step kept, from the underlying multi-agent system, where it applies
- ☐ A runnable test suite that would actually catch a real regression
- ☐ Structured logging + tracking, good enough to debug a bad production answer after the fact
- ☐ Basic rate limiting, retries, and timeouts on every outside call
- ☐ You can defend cost, scaling, and security trade-offs when pushed on them directly
- ☐ Meets a production-ready bar: correctness, safety, observability, and the ability to defend every design trade-off

Status

Not started. Track your own progress however works for you.